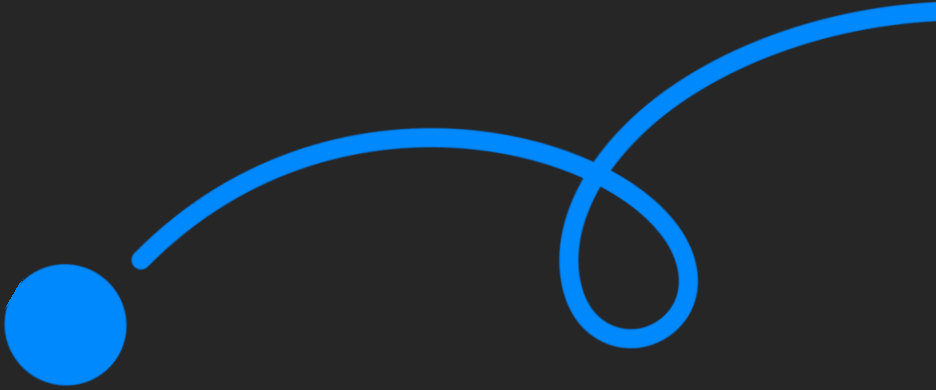




코멘토 실무PT 클래스

반도체 SW 엔지니어를 위한 새로운 SoC 위에 리눅스 디바이스 드라이버 포팅하기

멘티님, 반갑습니다!

클래스는 20:00에 시작됩니다.

출석 확인을 위해 프로필은 **실명**으로 설정해주세요.

출석을 확인합니다.



김대용

김진영

원종진

조현경

최승원

최민우

최강주

지난 세션은 어떠셨나요.

- 1 지난 주차에 어떤걸 공부했었는지 짧게 리뷰 해봐요.
- 2 복습을 진행하면서 궁금한 점이 있다면 같이 토의해봐요.
- 3 과제를 진행하면서 어려웠던 점을 알려주세요.
- 4 멘티 여러분이 작성한 과제를 다같이 살펴보도록 해요.

시작하기에 앞서

1

수업 진행을 하며
멘티님들의
이해도를 체크하며
진행할 예정입니다.

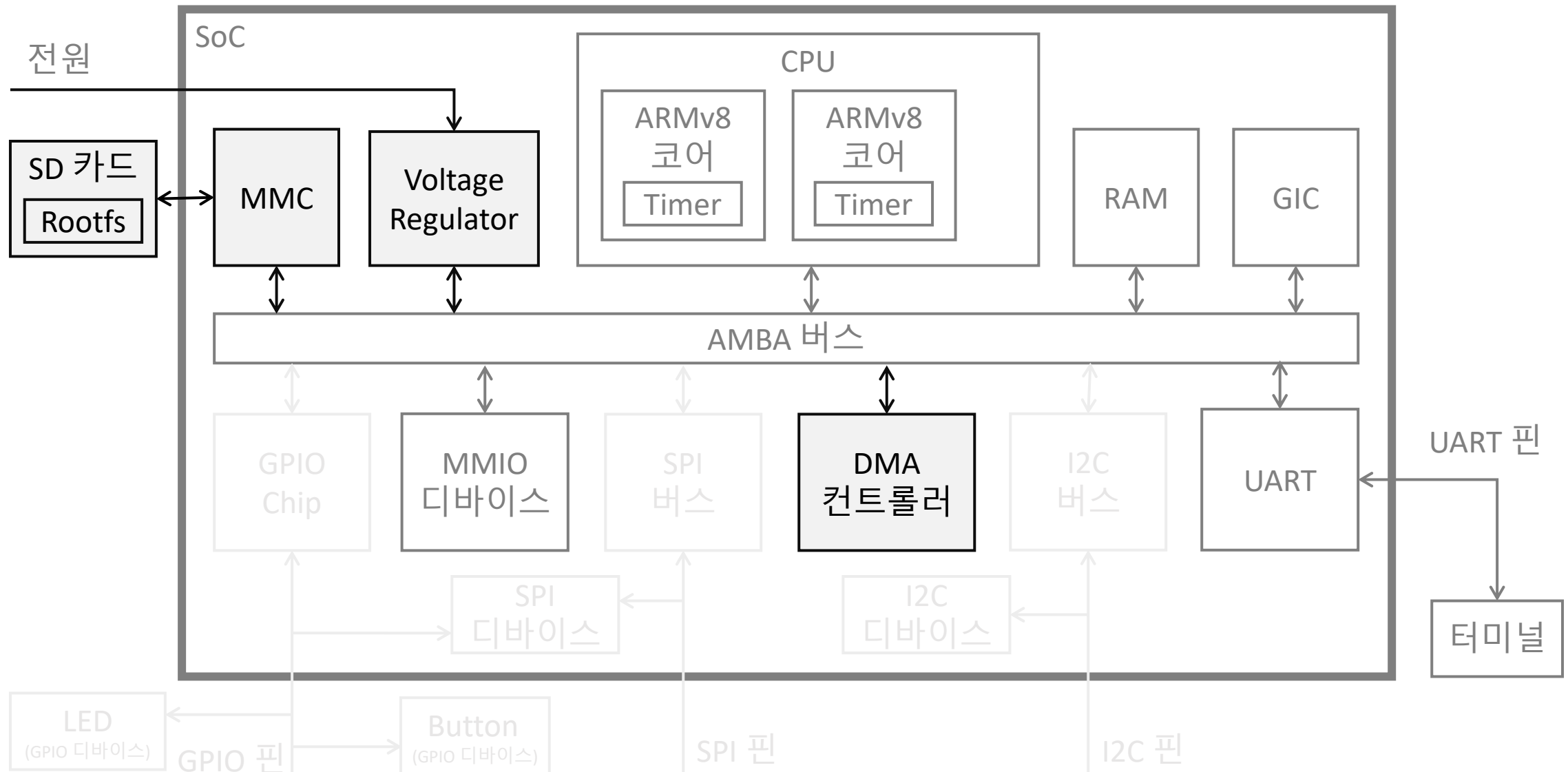
2

이해가 되지 않거나,
궁금한 내용은
채팅으로 언제든지
남겨주세요.

3

평균 50분 라이브를
진행하고, 10분의
쉬는 시간으로
진행 됩니다.

이번 수업에서 진행할 내용



01 DMA 개념 파악하기

02 Scatter/Gather 개념 파악하기

03 DMA 컨트롤러 추가하기

04 드라이버에서 DMA API 사용하기

05 MMC 하드웨어로 Rootfs 사용하기

Chapter. 1

DMA 개념 파악하기

대용량의 입출력 데이터 처리하기

지난 시간에 MMIO 하드웨어를 개발하였을 때 입출력 처리 방식은?

- 입력 : 데이터 레지스터를 1byte 씩 읽어서 문자열을 구성
- 출력 : 데이터 레지스터로 전달하고자 하는 문자열을 1byte씩 쓰기

```
while (len > 0) {
    bytes = MIN(len, sizeof(mmio_buf));
    bytes -= copy_from_user(mmio_buf, buf, bytes);
    for (i = 0; i < bytes; i++) {
        wait_event_interruptible(cmdev->tx_wq,
            !(readl(cmdev->base + COMENTO_FR) & COMENTO_FLAG_TXFF));
        writel(mmio_buf[i], cmdev->base + COMENTO_DR);
    }
    buf += bytes;
    len -= bytes;
}
```

```
for (i = 0, flag = readl(cmdev->base + COMENTO_FR);
    !(flag & COMENTO_FLAG_RXFE);
    flag = readl(cmdev->base + COMENTO_FR), i++) {
    cmdev->rx_buf[i] = readl(cmdev->base + COMENTO_DR);
}
cmdev->rx_size = i;
```

↑ 디바이스 코드 입력 부분

← 디바이스 코드 중 출력 부분

- 반복문을 돌면서 1byte씩 데이터 레지스터로 처리하면서 버퍼의 상태도 체크

➤ 만약 대용량의 데이터를 처리해야 한다면? 일일이 1byte씩 복사하느라 성능 저하가 발생합니다.

PIO (Programmed I/O) 란?

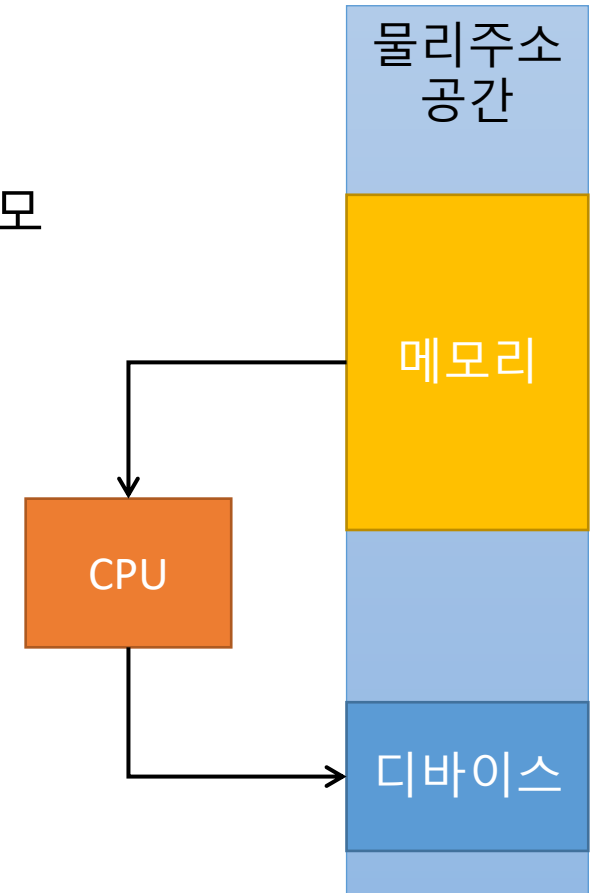
앞서 설명한 1byte씩 데이터를 처리하는 방식을 PIO라고 부름

반복문을 돌면서 데이터 레지스터로 데이터를 처리 → CPU 자원 소모

- 데이터를 디바이스로 쓰려면,
 1. 메모리의 내용을 CPU의 레지스터로 복사
 2. CPU 레지스터의 내용을 디바이스 레지스터로 복사

디바이스 버퍼의 상태를 확인 → CPU의 자원 소모

- 데이터를 처리하기 전에 버퍼의 상태를 확인하려면,
 1. 디바이스 레지스터의 내용을 CPU 레지스터로 복사
 2. CPU 레지스터에 있는 값과 AND 비트연산을 수행



DMA (Direct Memory Access) 란?

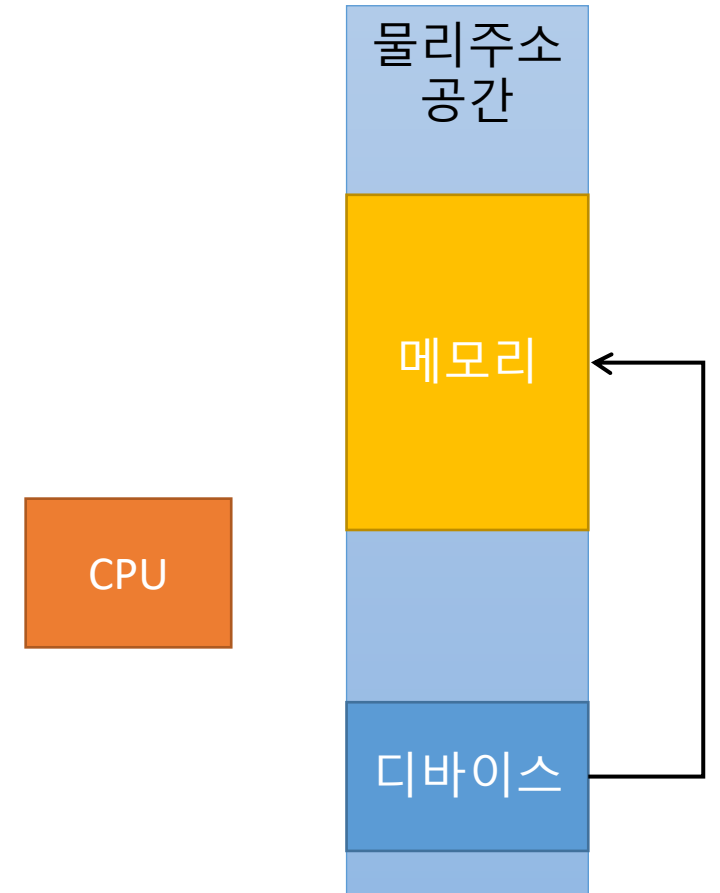
디바이스가 직접 메모리에 접근해서 데이터를 읽거나 쓰는 방식

- 데이터 처리에 더 이상 CPU의 자원을 사용하지 않아도 됨
 - CPU는 그 동안 다른 작업을 처리할 수 있으므로, 성능 향상
- 데이터 처리 완료 및 오류 발생시 CPU에게 통지
 - 디바이스가 인터럽트를 발생시키면 CPU에서 인터럽트 핸들러 실행 됨

메모리에 접근하는 기능을 추가해야 하므로 복잡한 구현 필요

- 메모리에 접근하는 디바이스 속도가 느리다면, 성능 저하
- 많은 디바이스가 메모리에 동시에 접근하게 되면, 성능 저하

➤ DMA 컨트롤러 (Controller)라는 전용의 하드웨어가 도입됩니다.



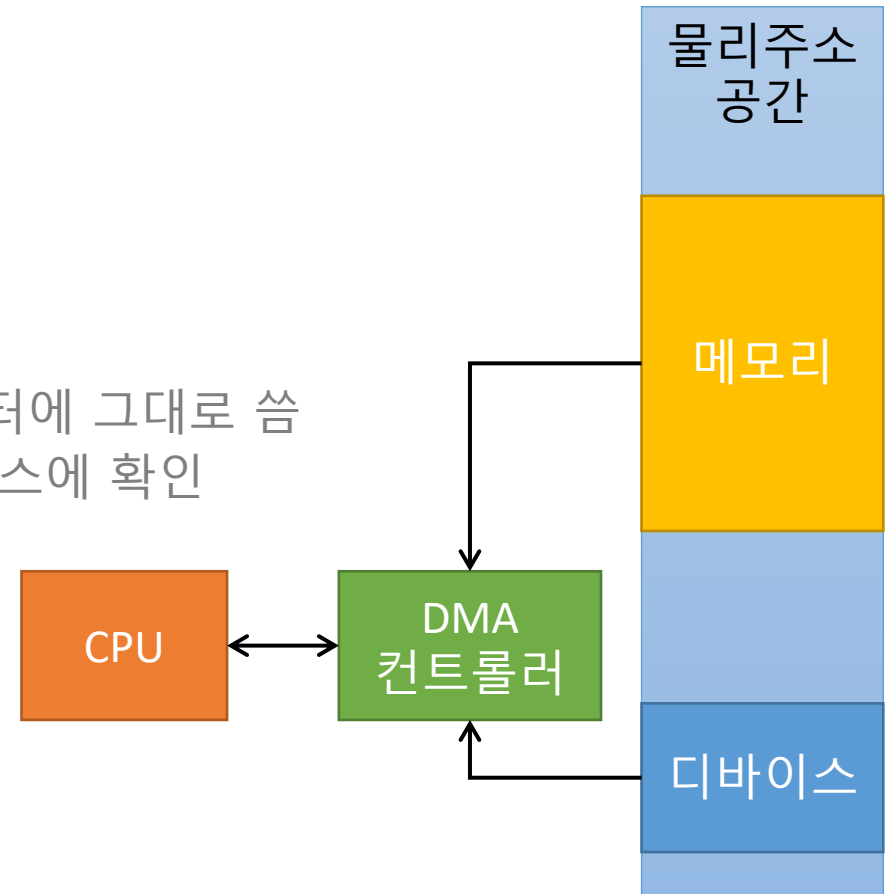
DMA 컨트롤러 (Controller) 란?

PIO 방식의 데이터 처리를 CPU 대신 해주는 하드웨어

- 데이터를 출력할 때,

1. CPU는 DMA 컨트롤러에게 데이터 출력을 요청
2. DMA 컨트롤러가 메모리의 내용을 읽음
3. 읽은 내용을 DMA 컨트롤러가 디바이스의 데이터 레지스터에 그대로 씀
4. 중간에 버퍼가 꽉 차지 않았는지 DMA 컨트롤러가 디바이스에 확인
5. DMA 컨트롤러는 인터럽트를 통해 CPU에 완료를 통보

데이터의 입력 역시 같은 흐름으로 이루어짐





질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.

Chapter. 2

Scatter / Gather 개념 파악하기

DMA 컨트롤러에서의 주소 공간

DMA 컨트롤러는 메모리에 접근할 때 물리 주소만을 사용

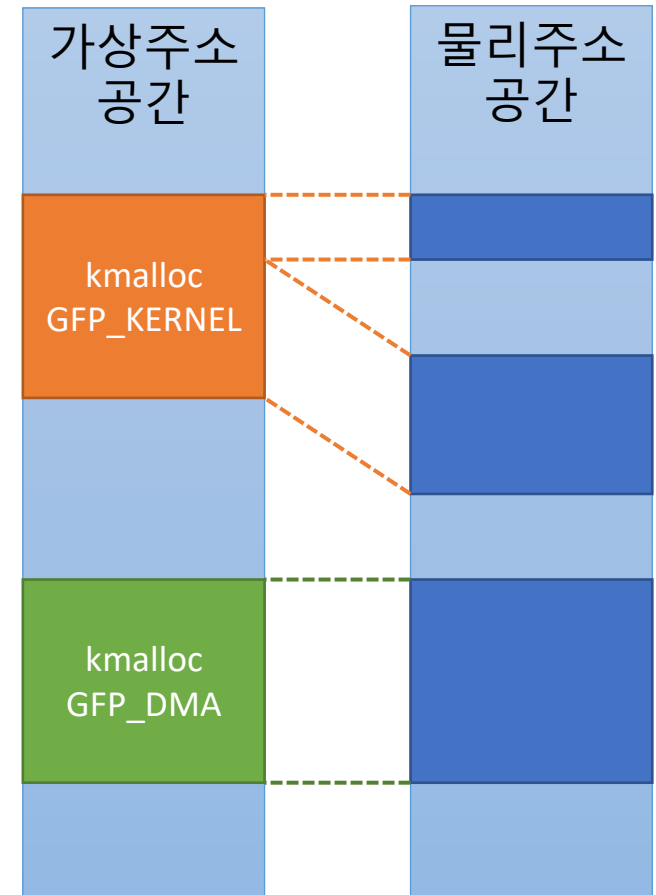
- 프로세스가 각각의 주소 공간을 사용하기 위해 가상주소를 사용
- 커널 모드도 가상 주소를 사용하여 메모리에 접근

커널에서 메모리를 할당하면 가상주소가 연속인 영역을 사용

- 물리주소 상으로는 연속되지 않을 수 있음
 - 가상주소만 사용하므로 가상주소의 연속여부가 중요
- DMA 컨트롤러를 위해서는 물리 주소상 연속된 메모리가 필요
 - kmalloc시 GFP_DMA만 물리 주소상으로 연속된 메모리를 할당

```
DMA: preallocated 128 KiB GFP_KERNEL pool for atomic allocations
DMA: preallocated 128 KiB GFP_KERNEL|GFP_DMA pool for atomic allocations
DMA: preallocated 128 KiB GFP_KERNEL|GFP_DMA32 pool for atomic allocations
```

- 커널 부팅시 물리 연속 공간 지원을 위해 GFP_DMA 메모리 풀은 따로 세팅



연속적인 공간의 할당

작은 크기(~ 4KB)의 물리주소를 연속되게 할당 하는 것은 어렵지 않음

- 물리주소를 쪼개서 관리할 때 최소 단위가 4KB이기 때문

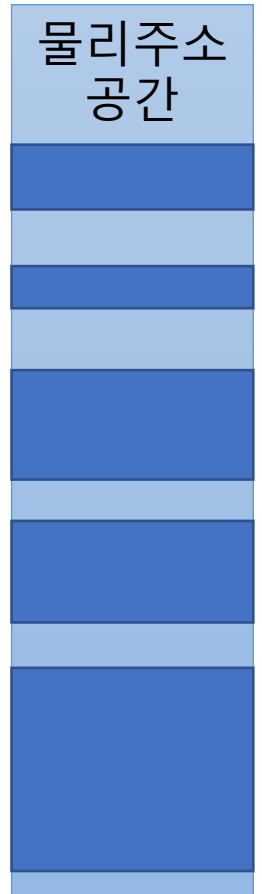
크기가 클 수록 물리주소를 연속되게 만드는 것은 어려움

- 메모리 영역을 할당 받고 해제하다 보면 파편화(Fragmentation) 발생
 - 비어 있는 메모리 영역이 쪼개져 있어, 빈 공간은 충분해도 연속할당은 할 수 없는 경우

다음과 같은 크기의 영역을 할당 받을 때,

필요한 크기

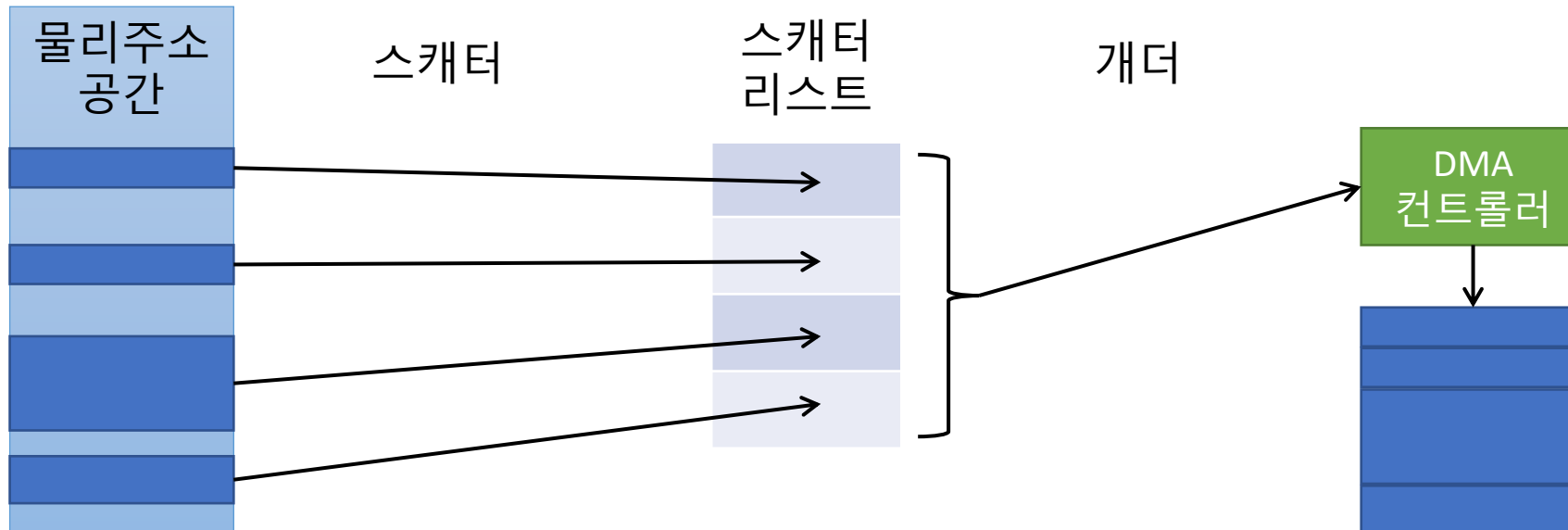
여러 빈 영역을 합쳐서 연속된 가상 주소를 만드는 것은 가능
연속된 물리 주소를 할당하는 것은 불가능



스캐터(Scatter)와 개더(Gather)란?

연속되어 있지 않은 물리 주소를 DMA에서 사용하기 위한 기법

- 스캐터 (Scatter) – 여러 개의 메모리 영역으로 쪼개어 스캐터 리스트 (scatter list)에 관리
- 개더 (Gather) – 스캐터 리스트에 있는 영역들을 하나의 큰 영역인 것처럼 데이터 처리

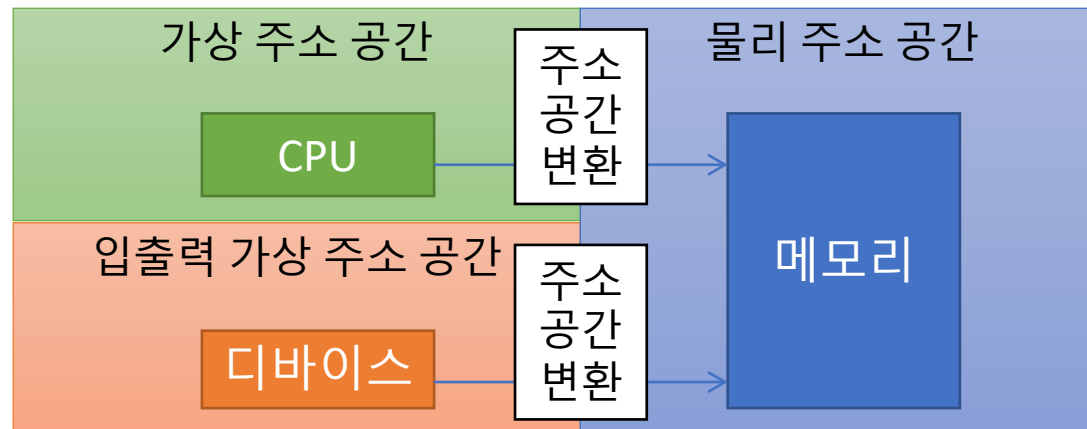


IOVA (I/O Virtual Address, 입출력 가상주소) 란?

스캐터-개더 기법은 스캐터 리스트를 분석해야 하므로 구현이 복잡하고 성능이 떨어짐

물리주소로 연속되지 않은 공간을 연속된 것 처럼 사용하기 위해 별도의 주소 공간 도입

- 디바이스를 위한 별도의 주소공간 – IOVA (I/O Virtual Address 입출력 가상 주소)



➤ IOVA는 어려운 고급 기술이므로 본 강의에서는 개념만 파악하고 넘어갑니다.



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.



쉬는 시간!

조금 쉬었다가, 다시 만나요!

Chapter. 3

DMA 컨트롤러 추가하기

QEMU의 새 SoC에 DMA 컨트롤러 추가하기

```
...
struct ComentoMachineState {
    ...
    DeviceState *dma;
};
...
#define COMENTO_IRQ_DMA 15
...
#define COMENTO_BASE_DMA 0x09012000
...

static void create_dma(ComentoMachineState *vms, MemoryRegion *mem)
{
    hwaddr base = COMENTO_BASE_DMA;
    int irq = COMENTO_IRQ_DMA;
    SysBusDevice * busdev;
    int i;

    // ARM PL330 DMA 컨트롤러 장치 추가
    vms->dma = qdev_new("pl330");

    // DMA 컨트롤러가 접근할 메모리를 설정
    object_property_set_link(OBJECT(vms->dma), "memory", OBJECT(mem),
                             &error_fatal);

    busdev = SYS_BUS_DEVICE(vms->dma);
    sysbus_realize_and_unref(busdev, &error_fatal);
    sysbus_mmio_map(busdev, 0, base);
}
```

```
// DMA 컨트롤러에서 오류가 발생했을 때 통지할 인터럽트 할당
sysbus_connect_irq(busdev, 0, qdev_get_gpio_in(vms->gic, irq));

// DMA 컨트롤러에서 발생하는 이벤트를 처리하기 위한 인터럽트 할당
for (i = 0; i < 16; i++) {
    sysbus_connect_irq(busdev, i + 1,
                      qdev_get_gpio_in(vms->gic, irq + i + 1));
}
//DMA 컨트롤러에는 총 1 + 16, 즉 17개의 인터럽트가 할당되게 됨
}

...

static void machcomento_init(MachineState *machine)
{
    ...
    create_dma(vms, sysmem);

    // MMIO 하드웨어에서 DMA를 사용하도록 성능개선 할 예정이므로
    // MMIO 하드웨어 이전에 DMA 컨트롤러가 초기화 되어야 함
    create_mmio(vms, sysmem);
}
```

<hw/arm/comento.c>

디바이스 트리와 커널 설정에 DMA 컨트롤러 추가하기

```
...
dma: pl330@9014000 {
    compatible = "arm,pl330", "arm,primecell";
    clock-names = "apb_pclk"; // AMBA 장치이므로 클럭 필요
    clocks = <&clock>;
    // DMA 컨트롤러에서 가지고 있는 인터럽트는 17개
    interrupts = <GIC_SPI 15 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 16 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 17 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 18 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 19 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 20 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 21 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 22 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 23 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 24 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 25 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 26 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 27 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 28 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 29 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 30 IRQ_TYPE_LEVEL_HIGH>,
                <GIC_SPI 31 IRQ_TYPE_LEVEL_HIGH>;
    reg = <0x00 0x9012000 0x00 0x1000>; // MMIO로 사용할 주소 명시
    // DMA 컨트롤러를 참조할 때 숫자는 1칸만 사용
    #dma-cells = <1>; // 여기서의 1칸은 integer 크기를 나타냄
};
...
```

```
...
CONFIG_DMADEVICES=y
CONFIG_DMA_ENGINE=y
CONFIG_PL330_DMA=y
```

<arch/arm64/configs/comento_defconfig>

수정한 defconfig를 적용하기 위해
make comento_defconfig 명령어 사용

<arch/arm64/boot/dts/comento/comento.dts>

UART와 DMA 컨트롤러 연동하기

수업에서 사용하고 있는 ARM PL011 UART 하드웨어는 DMA 방식을 지원

- 하지만, QEMU에서는 구현이 완전히 되어 있지 않아 송신 부분만 DMA 사용 가능

리눅스의 ARM PL011 UART 드라이버에서는 기본적으로 DMA 방식을 지원

- 아래와 같이 디바이스 트리에 DMA 설정을 추가하면 드라이버가 DMA로 동작

```
...
    pl011@9010000 {
        // 수신 부분은 QEMU에서 DMA 지원이 안되므로 송신 부분만 설정
        dma-names = "tx";
        // 송신 부분, 수신 부분 각각 DMA를 설정해야 함
        // DMA 컨트롤러의 3번 하드웨어를 송신을 위해 사용
        // DMA 컨트롤러의 #dma-cells를 1로 명시했으므로 숫자는 1칸 사용
        dmas = <&dma 3>;
    };
...
```

QEMU에서 DMA 동작 확인 실습

부팅될 때 DMA 컨트롤러인 ARM PL330이 초기화됨

- 컨트롤러가 데이터를 임시로 저장하기 위해 사용하는 버퍼의 크기는 $256 \times 8 \text{ bytes} = 2\text{KB}$
- 8개 채널, 4개의 하드웨어, 16개의 이벤트 지원
 - 본 수업에서는 4개의 하드웨어 중 2개를 MMIO의 송수신 부분 나머지 1개를 UART에 사용

```
dma-pl330 9012000.pl330: Loaded driver for PL330 DMAC-241330
dma-pl330 9012000.pl330:          DBUFF-256x8bytes Num_Chans-8 Num_Peri-4 Num_Events-16
```

UART에서도 DMA 채널을 초기화하여 송신 부분에 사용

- 디바이스 트리에 추가한 대로 DMA의 3번째 하드웨어 사용
 - 채널이라고 나와 있으나 ARM PL330에서는 채널이 아닌 하드웨어를 의미

```
uart-pl011 9010000.pl011: DMA channel TX dma0chan3
```


UART에서 DMA 동작하는 방식 살펴보기 (1/2)

UART 드라이버 내에 DMA 관련된 구현 포함됨

- drivers/tty/serial/amba-pl011.c

1. 디바이스 트리에 DMA 설정이 있는지 체크

- pl011_dma_probe

- ① dma_request_chan : 디바이스 트리에서 "tx"로 지정한 DMA의 채널 요청
 - 만약 DMA 장치를 초기화하는 게 실패했다면 (-EPROBE_DEFER), DMA 초기화 중단
- ② dmaengine_slave_config : DMA 채널 설정
 - dst_addr : UART 디바이스의 데이터 레지스터 주소
 - dst_addr_width : 1바이트씩 데이터 레지스터에 쓰도록 지정
 - direction : 메모리에서 데이터 레지스터로 데이터 전송

2. DMA에서 사용할 스캐터 리스트 초기화

- pl011_dma_startup

- sg_init_one : 스캐터 리스트에 버퍼 하나만 추가하여 초기화

```
static void pl011_dma_probe(struct uart_amba_port *uap)
{
    /* DMA is the sole user of the platform data right now */
    struct amba_pl011_data *plat = dev_get_platdata(uap->port.dev);
    struct device *dev = uap->port.dev;
    struct dma_slave_config tx_conf = {
        .dst_addr = uap->port.mapbase +
                    pl011_reg_to_offset(uap, REG_DR),
        .dst_addr_width = DMA_SLAVE_BUSWIDTH_1_BYTE,
        .direction = DMA_MEM_TO_DEV,
        .dst_maxburst = uap->fifo_size >> 1,
        .device_fc = false,
    };
    struct dma_chan *chan;
    dma_cap_mask_t mask;

    uap->dma_probed = true;
    chan = dma_request_chan(dev, "tx");
    if (IS_ERR(chan)) {
        if (PTR_ERR(chan) == -EPROBE_DEFER) {
            uap->dma_probed = false;
            return;
        }
    }
    ...
}
```

```
    dmaengine_slave_config(chan, &tx_conf);
    uap->dmatx.chan = chan;

    dev_info(uap->port.dev, "DMA channel TX %s\n",
             dma_chan_name(uap->dmatx.chan));
```

```
sg_init_one(&uap->dmatx.sg, uap->dmatx.buf, PL011_DMA_BUFFER_SIZE);
```

UART에서 DMA 동작하는 방식 살펴보기 (2/2)

2. DMA를 사용하여 데이터 전송

- pl011_dma_tx_refill
 - ① dma_map_sg : 스캐터 리스트를 주소 공간에 추가
 - ② dmaengine_prep_slave_sg : 스캐터 리스트 설정
 - ③ dmaengine_submit : DMA 컨트롤러에 DMA 작업 등록
 - ④ dma_async_issue_pending : 등록된 작업 시작

```
dma_tx->sg.length = count;
if (dma_map_sg(dma_dev->dev, &dma_tx->sg, 1, DMA_TO_DEVICE) != 1) {
    uap->dma_tx.queued = false;
    dev_dbg(uap->port.dev, "unable to map TX DMA\n");
    return -EBUSY;
}
desc = dmaengine_prep_slave_sg(chan, &dma_tx->sg, 1, DMA_MEM_TO_DEV,
                               DMA_PREP_INTERRUPT | DMA_CTRL_ACK);
if (!desc) {
    dma_unmap_sg(dma_dev->dev, &dma_tx->sg, 1, DMA_TO_DEVICE);
    uap->dma_tx.queued = false;
    /*
     * If DMA cannot be used right now, we complete this
     * transaction via IRQ and let the TTY layer retry.
     */
    dev_dbg(uap->port.dev, "TX DMA busy\n");
    return -EBUSY;
}
```

...

```
/* All errors should happen at prepare time */
dmaengine_submit(desc);

/* Fire the DMA transaction */
dma_dev->device_issue_pending(chan);
```

Chapter. 4

드라이버에서 DMA API를 사용하기

DMA 컨트롤러를 사용하여 성능 개선하기

DMA 컨트롤러를 사용하면 PIO로 구현된 부분 개선 가능

- 현재 버퍼의 상태를 체크하고 데이터 레지스터에 복사하는 부분을 DMA 컨트롤러에 위임
- DMA 컨트롤러가 작업을 완료하면 인터럽트로 CPU에 통지

송신시 버퍼가 꽉 차있거나, 수신시 버퍼가 비어있으면 일시적으로 작업 지연 필요

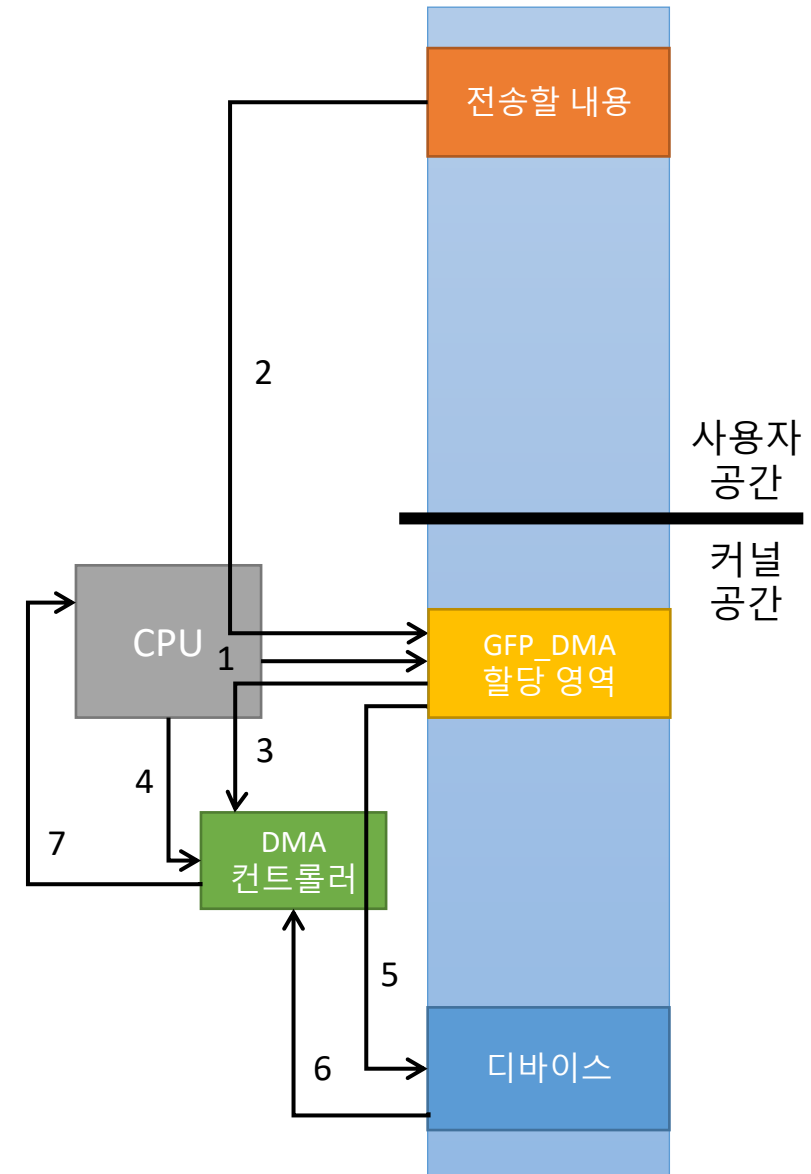
- 이러한 지연을 흐름 제어 (Flow control)라고 함
- 일반적인 DMA 컨트롤러에는 이러한 제어를 하기 위한 구조가 마련되어 있음
 - ARM PL330 실제 하드웨어에서는 조금 더 복잡한 방식을 사용 QEMU에서는 내부신호 하나로 간략화

수신시 언제 DMA를 중단할 수 있을지 시점 고려 필요

- 폴링 : 주기적으로 수신할 데이터가 있는지 확인하고 일정 시간 이상 없으면 DMA 중단
 - 인터럽트 추가 : 수신할 데이터가 없을 때 인터럽트를 발생시키고, 인터럽트가 발생하면 DMA 중단
- 본 수업에서는 새로운 인터럽트를 추가하는 방식으로 DMA 컨트롤러를 사용해 보시다.

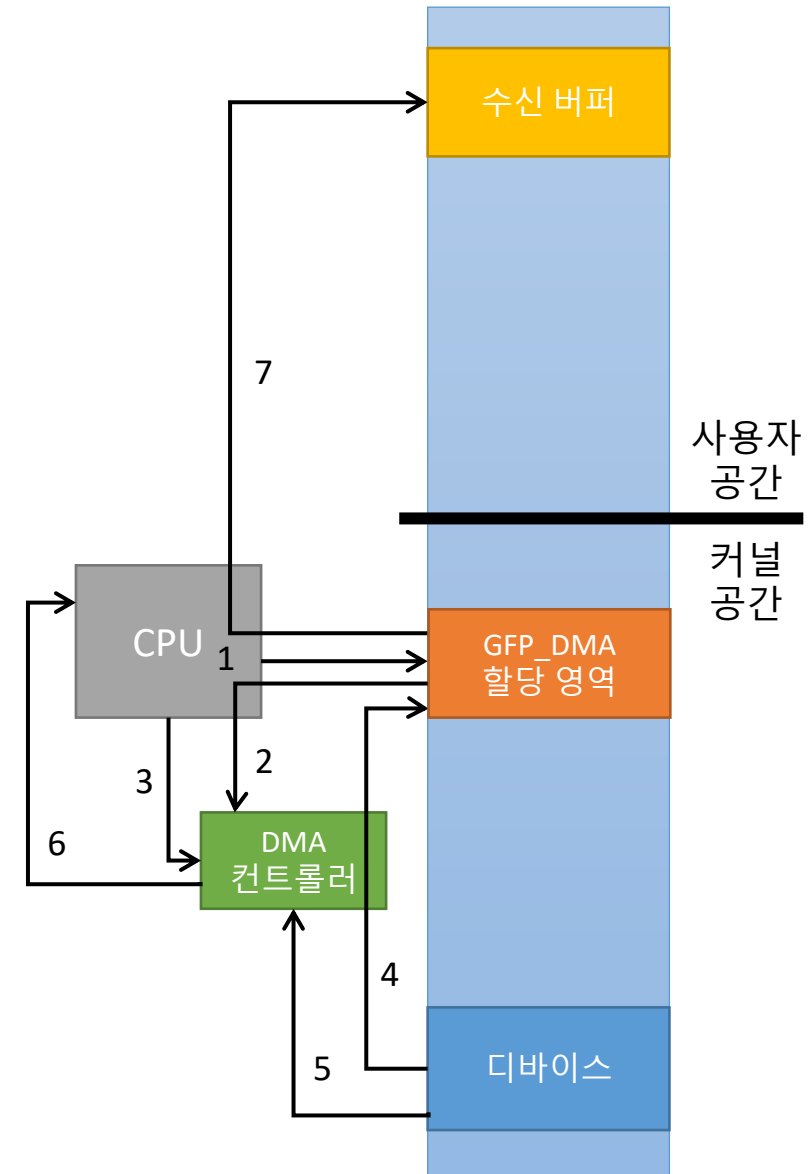
DMA 컨트롤러를 사용한 전송 기능 구현

1. 메모리를 GFP_DMA 타입을 줘서 kmalloc으로 할당
2. copy_from_user로 전송할 내용을 할당 받은 영역으로 복사
3. 할당 받은 영역을 스캐터 리스트로 만들고 DMA 컨트롤러에 설정
4. DMA 컨트롤러가 작업하도록 시작
5. DMA 컨트롤러는 할당 받은 영역에서 데이터 레지스터로 복사
6. 버퍼가 꽉 찼을 때, 디바이스는 DMA 컨트롤러에 흐름 제어 신호
7. 5~6을 반복하다가 작업이 완료되면 DMA 컨트롤러는 인터럽트로 통지



DMA 컨트롤러를 사용한 수신 기능 구현

1. 메모리를 GFP_DMA 타입을 줘서 kmalloc으로 할당
2. 할당 받은 영역을 스캐터 리스트로 만들고 DMA 컨트롤러에 설정
3. DMA 컨트롤러가 작업하도록 시작
4. DMA 컨트롤러는 데이터 레지스터에서 할당 받은 영역으로 복사
5. 버퍼가 비었을 때, 디바이스는 DMA 컨트롤러에 흐름 제어 신호
6. 디바이스는 버퍼가 비었음을 인터럽트로도 통지
7. DMA를 중단하고 copy_to_user로 수신한 내용을 사용자 공간으로 복사



QEMU에서 DMA컨트롤러와 MMIO 하드웨어 연결하기

```
#define COMENTO_INT_RX_EMPTY    (1 << 2)
...
struct MMIO_COMENTO_State {
    ...
    qemu_irq dma_tx_busy;
    qemu_irq dma_rx_busy;
};
...
static uint64_t comento_mmio_read(void *opaque, hwaddr offset,
                                unsigned size)
{
    ...
    case COMENTO_DR:
        if (comento_fifo_is_empty(&s->receive)) {
            ...
            s->ris != COMENTO_INT_RX_EMPTY;
            qemu_set_irq(s->dma_rx_busy, 1);
        }
    ...
    static void comento_mmio_write(void * opaque, hwaddr offset,
                                uint64_t value, unsigned size)
    {
        ...
        case COMENTO_DR:
            ...
            if (comento_fifo_is_full(&s->transmit)) {
                ...
                qemu_set_irq(s->dma_tx_busy, 1);
            }
        ...
    }
```

<hw/misc/comento/mmio.c>

```
static char *comento_get_data(Object *obj, Error **errp)
{
    ...
    qemu_set_irq(s->dma_tx_busy, 0);
    return ret;
}
static void comento_set_data(Object *obj, const char *value, Error **errp)
{
    ...
    s->ris &= ~COMENTO_INT_RX_EMPTY;
    qemu_set_irq(s->dma_rx_busy, 0);
    comento_mmio_update(s);
}
...
static void comento_mmio_init(Object *obj)
{
    ...
    sysbus_init_irq(dev, &s->dma_tx_busy);
    sysbus_init_irq(dev, &s->dma_rx_busy);
    ...
}
```

```
static void create_mmio(const ComentoMachineState *vms, MemoryRegion *mem)
{
    ...
    //호름 제어를 위한 신호를 mmio의 1(송신), 2(수신)번 핀에 할당하고
    //DMA 컨트롤러의 0, 1 하드웨어에 연결
    sysbus_connect_irq(s, 1, qdev_get_gpio_in(vms->dma, 0));
    sysbus_connect_irq(s, 2, qdev_get_gpio_in(vms->dma, 1));
}
```

<hw/arm/comento.c>

리눅스에서 DMA 채널 할당 받기 (1/2)

```
...
#include <linux/dmaengine.h>
#include <linux/dma-mapping.h>
...
#define COMENTO_DMA_BUFFER_SIZE 4096

struct comento_dma_buf {
    struct dma_chan *chan; // DMA 채널
    struct scatterlist sg; // 스캐터 리스트
    dma_cookie_t cookie; // DMA를 중단하기 위한 핸들
    char *buf; // GFP_DMA로 할당 받은 영역
    bool queued; // 현재 DMA 동작 중인지 아닌지
};

struct comento_mmio_device {
    ...
    struct comento_dma_buf tx;
    struct comento_dma_buf rx;
    struct work_struct rx_dma_work; // 더 이상 수신할 데이터가 없어서
    // 인터럽트를 발생된 경우, DMA를 중단하기 위한 워크 큐 (Bottom half)
    ...
};

static int comento_dma_init(struct device *dev, const char *name,
                           struct dma_slave_config *conf, struct comento_dma_buf *dma)
{
    int ret; // 디바이스 트리에 정의된
    dma->chan = dma_request_chan(dev, name); // DMA 채널 요청
    if (IS_ERR(dma->chan)) {
        ret = PTR_ERR(dma->chan);
        goto err_req;
    }
    printk(KERN_INFO "Comento: DMA%s%s\n", name, dma_chan_name(dma->chan));
```

```
dmaengine_slave_config(dma->chan, conf); // DMA 채널 설정

//GFP_DMA 타입으로 할당 받은 후, 스캐터 리스트 생성
dma->buf = dev_kmalloc(dev, COMENTO_DMA_BUFFER_SIZE,
                      GFP_KERNEL | GFP_DMA);
sg_init_one(&dma->sg, dma->buf, COMENTO_DMA_BUFFER_SIZE);
dma->queued = false; // DMA 동작 중 아님으로 설정
return 0;

err_req:
    return ret;
}

static int comento_dma_probe(struct amba_device *adev,
                             struct comento_mmio_device *cmdev)
{
    int ret;
    unsigned long flag;
    struct dma_slave_config tx_conf = { // 송신 부분 설정
        .dst_addr = adev->res.start + COMENTO_DR,
        .dst_addr_width = DMA_SLAVE_BUSWIDTH_1_BYTE,
        .direction = DMA_MEM_TO_DEV,
        .dst_maxburst = 1, // 한 번에 최대 1바이트만 읽도록 설정
        .device_fc = false,
    };
    struct dma_slave_config rx_conf = {
        .src_addr = adev->res.start + COMENTO_DR,
        .src_addr_width = DMA_SLAVE_BUSWIDTH_1_BYTE,
        .direction = DMA_DEV_TO_MEM, // 수신 부분 설정
        .src_maxburst = 1,
        .device_fc = false,
    };
};
```


리눅스에서 DMA 채널 할당 받기 (2/2)

```
ret = comento_dma_init(&adev->dev, "tx", &tx_conf, &cmdev->tx);
if (ret) {
    printk(KERN_ERR "Comento: unable to initialize DMA tx\n");
    return ret;
}
ret = comento_dma_init(&adev->dev, "rx", &rx_conf, &cmdev->rx);
if (ret) {
    printk(KERN_ERR "Comento: unable to initialize DMA rx\n");
    dma_release_channel(cmdev->rx.chan);
    return ret;
}
// 수신 버퍼가 비었을 때 인터럽트를 받도록 설정
flag = readl(cmdev->base + COMENTO_IMSC);
writel(flag | COMENTO_INT_RX_EMPTY, cmdev->base + COMENTO_IMSC);

INIT_WORK(&cmdev->rx_dma_work, comento_mmio_rx_dma_work_handler);
return 0;
}
...
static void comento_mmio_remove(struct amba_device *adev)
{
    if (cmdev->dev) {
        ...
        dma_release_channel(cmdev->rx.chan); // MMIO 장치가 해제될 때
        dma_release_channel(cmdev->tx.chan); // 할당한 DMA 채널도 해제
    }
    ...
}
```

```
...
static int comento_mmio_probe(struct amba_device *adev,
                             const struct amba_id *id)
{
    ret = comento_dma_probe(adev, cmdev);
    if (ret) {
        goto out;
    }
    ...
}
```

<drivers/comento/mmio.c>

```
...
mmio@9020000 {
    ...
    // dma-names의 이름으로 dmas에 설정된 DMA 하드웨어 사용
    // DMA 컨트롤러의 0번을 하드웨어를 송신 부분으로
    // 1번 하드웨어를 수신으로 사용
    dma-names = "tx", "rx";
    dmas = <&dma 0x0>, <&dma 0x1>;
};
...
```

<arch/arm64/boot/dts/comento/comento.dts>

리눅스에서 DMA로 송신 부분 개선하기 (1/2)

```
static void comento_dma_tx_callback(void *data)
{
    struct comento_mmio_device *cmdev = (struct comento_mmio_device *)data;

    // 스캐터 리스트 주소공간에서 제거
    dma_unmap_sg(cmdev->dev->parent, &cmdev->tx.sg, 1, DMA_TO_DEVICE);
    // DMA 동작 중이 아님으로 변경
    cmdev->tx.queued = false;
    // DMA가 동작 중 아님으로 바뀌길 원하는 대기 큐 깨우기
    wake_up_interruptible(&cmdev->tx_wq);
}

// MMIO 하드웨어 드라이버의 송신부분을 DMA 사용하여 다시 작성해야 함
static ssize_t comento_mmio_write(struct file *fp, const char __user *buf,
                                size_t len, loff_t *ppos)
{
    struct comento_mmio_device *cmdev;
    ssize_t written_bytes = 0, bytes;
    struct dma_async_tx_descriptor *desc;

    cmdev = comento_mmio_get_device(iminor(fp->f_inode));
    if (cmdev == NULL) {
        return written_bytes;
    }

    while (len > 0) {
        // DMA가 동작중이면 끝날때 까지 대기
        wait_event_interruptible(cmdev->tx_wq, !cmdev->tx.queued);

        bytes = MIN(len, COMENTO_DMA_BUFFER_SIZE);
```

```
        // 사용자 공간에 있는 데이터를 커널로 복사
        bytes -= copy_from_user(cmdev->tx.buf, buf, bytes);
        // 스캐터 리스트 주소 공간에 추가
        cmdev->tx.sg.length = bytes;
        if (dma_map_sg(cmdev->dev->parent, &cmdev->tx.sg, 1,
                      DMA_TO_DEVICE) != 1) {
            printk(KERN_ERR "Comento: unable to map DMA tx\n");
            break;
        }

        // DMA 컨트롤러에 스캐터 리스트 설정
        desc = dmaengine_prep_slave_sg(cmdev->tx.chan, &cmdev->tx.sg, 1,
                                       DMA_MEM_TO_DEV,
                                       DMA_PREP_INTERRUPT | DMA_CTRL_ACK);

        if (!desc) {
            printk(KERN_ERR "Comento: DMA tx prep error\n");
            dma_unmap_sg(cmdev->dev->parent, &cmdev->tx.sg, 1,
                          DMA_TO_DEVICE);
            break;
        }

        // DMA 작업이 완료되었을 때 호출될 함수 설정
        desc->callback = comento_dma_tx_callback;
        desc->callback_param = cmdev;
        // DMA 동작 중으로 설정
        cmdev->tx.queued = true;

        // DMA 작업 등록
        cmdev->tx.cookie = dmaengine_submit(desc);

        // 등록한 DMA 작업 바로 시작
        dma_async_issue_pending(cmdev->tx.chan);
```

리눅스에서 DMA로 송신 부분 개선하기 (2/2)

```
        buf += bytes;
        len -= bytes;
        written_bytes += bytes;
    }
    // DMA 작업이 완료되어 DMA가 동작 중 아님으로 바뀔 때까지 기다림
    wait_event_interruptible(cmdev->tx_wq, !cmdev->tx.queued);
    *ppos += written_bytes;
    return written_bytes;
}

//MMIO 하드웨어 드라이버의 인터럽트 부분 역시 다시 작성 필요
static irqreturn_t comento_mmio_interrupt(int irq, void *dev_id)
{
    struct comento_mmio_device *cmdev = dev_id;
    unsigned long mis;

    mis = readl(cmdev->base + COMENTO_MIS);
    if (mis & COMENTO_INT_TX) {
        writel(COMENTO_INT_TX, cmdev->base + COMENTO_ICR);
        // 기존 코드에서는 송신 내용이 모두 처리되어 인터럽트가 발생할 때,
        // comento_mmio_write에서 버퍼가 꽉 차서 기다리고 있었다면
        // 해당 대기 큐를 깨우는 코드가 있었음
        // DMA에서는 DMA 동작 중이 아님으로 바뀔 때까지 기다리는 코드가
        // 추가되었으므로 더 이상 필요하지 않음
        return IRQ_HANDLED;
    }
    return IRQ_NONE;
}
```

리눅스에서 DMA로 수신 부분 개선하기 (1/2)

```
static void comento_dma_rx_callback(void *data)
{
    struct comento_mmio_device *cmdev = (struct comento_mmio_device *)data;
    // 스캐터 리스트 주소공간에서 제거
    dma_unmap_sg(cmdev->dev->parent, &cmdev->rx.sg, 1, DMA_FROM_DEVICE);
    // DMA 동작 중이 아님으로 변경
    cmdev->rx.queued = false;
}

// 수신 작업은 인터럽트가 발생했을 때 워크 큐로 처리됨
// MMIO 하드웨어 드라이버에서는 수신 데이터가 들어와 인터럽트 발생했을 때
// 워크 큐를 사용 하여 수신 작업을 진행하였음 (Bottom half)
// 이러한 수신 부분 역시 다시 작성 필요
static void comento_mmio_rx_work_handler(struct work_struct *work)
{
    struct dma_async_tx_descriptor *desc;
    struct comento_mmio_device *cmdev = container_of(work,
        struct comento_mmio_device, rx_work);

    write_lock(&cmdev->lock);
    // 만약 DMA가 동작 중이라면, 이전 동작은 중단 시킴
    if (cmdev->rx.queued) {
        dmaengine_terminate_sync(cmdev->rx.chan);
        comento_dma_rx_callback(cmdev); // DMA 완료 함수는 직접 호출
    }
    // 스캐터 리스트 주소 공간에 추가
    if (dma_map_sg(cmdev->dev->parent, &cmdev->rx.sg, 1,
        DMA_FROM_DEVICE) != 1) {
        printk(KERN_ERR "Comento: unable to map DMA rx\n");
        goto err;
    }
}
```

```
// DMA 컨트롤러에 스캐터 리스트 설정
desc = dmaengine_prep_slave_sg(cmdev->rx.chan, &cmdev->rx.sg, 1,
    DMA_DEV_TO_MEM,
    DMA_PREP_INTERRUPT | DMA_CTRL_ACK);

if (!desc) {
    printk(KERN_ERR "Comento: DMA rx prep error\n");
    dma_unmap_sg(cmdev->dev->parent, &cmdev->rx.sg, 1,
        DMA_FROM_DEVICE);

    goto err;
}
// DMA 작업이 완료되었을 때 호출될 함수 설정
desc->callback = comento_dma_rx_callback;
desc->callback_param = cmdev;
cmdev->rx.queued = true; // DMA 동작 중으로 설정

cmdev->rx.cookie = dmaengine_submit(desc); // DMA 작업 등록
// 등록된 DMA 작업 바로 시작
dma_async_issue_pending(cmdev->rx.chan);
err:
    write_unlock(&cmdev->lock);
}
//MMIO 하드웨어 드라이버의 인터럽트 부분 역시 다시 작성 필요
static irqreturn_t comento_mmio_interrupt(int irq, void *dev_id)
{
    ...
    if (mis & COMENTO_INT_RX) { // 수신 데이터가 들어와 인터럽트 발생
        writel(COMENTO_INT_RX, cmdev->base + COMENTO_ICR);
        // 워크 큐로 comento_mmio_rx_work_handler 실행
        schedule_work(&cmdev->rx_work);
        return IRQ_HANDLED;
    }
}
```

리눅스에서 DMA로 수신 부분 개선하기 (2/2)

```
// 더 이상 수신 데이터가 없는 시점을 잡기 위해 새로 추가한 인터럽트
if (mis & COMENTO_INT_RX_EMPTY) {
    writel(COMENTO_INT_RX_EMPTY, cmdev->base + COMENTO_ICR);
    // 워크 큐로 comento_mmio_rx_dma_work_handler 실행
    schedule_work(&cmdev->rx_dma_work);
    return IRQ_HANDLED;
}
return IRQ_NONE;
}

// 더 이상 수신 데이터가 없는 경우 DMA 작업을 중단시켜
// DMA가 불필요하게 동작하지 않도록 추가
static void comento_mmio_rx_dma_work_handler(struct work_struct *work)
{
    struct comento_mmio_device *cmdev = container_of(work,
        struct comento_mmio_device, rx_dma_work);
    struct dma_tx_state state;

    // DMA 작업을 일시 정지 시킴
    dmaengine_pause(cmdev->rx.chan);
    // 완전히 중단시키지 않는 이유는
    // 여태까지 몇 바이트를 받았는지 상태를 받아오기 위함
    dmaengine_tx_status(cmdev->rx.chan, cmdev->rx.cookie, &state);
    // 여태까지 받은 크기 = 스캐터리스트 크기 - DMA 작업 남은 바이트 수
    cmdev->rx_size = cmdev->rx.sg.length - state.residue;
    // 만약 DMA가 동작 중이라면, DMA 작업을 완전히 중단시킴
    if (cmdev->rx.queued) {
        dmaengine_terminate_sync(cmdev->rx.chan);
        comento_dma_rx_callback(cmdev); // DMA 완료 함수는 직접 호출
    }
}
```

```
// MMIO 하드웨어 드라이버와 코드는 거의 동일함
static ssize_t comento_mmio_read(struct file *fp, char __user *buf,
    size_t len, loff_t *ppos)
{
    struct comento_mmio_device *cmdev;
    ssize_t read_bytes = 0;

    cmdev = comento_mmio_get_device(iminor(fp->f_inode));
    if (cmdev == NULL) {
        return read_bytes;
    }

    read_lock(&cmdev->lock);
    if (cmdev->rx_size <= len + *ppos) {
        len = cmdev->rx_size - *ppos;
    }
    // 유일하게 다른 부분으로
    // 기존의 cmdev->rx_buf를 cmdev->rx.buf로 변경해주면 됨
    read_bytes = len - copy_to_user(buf, cmdev->rx.buf + *ppos, len);
    *ppos += read_bytes;

    read_unlock(&cmdev->lock);

    return read_bytes;
}
```



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.



쉬는 시간!

조금 쉬었다가, 다시 만나요!

Chapter. 5

MMC 하드웨어로 Rootfs 사용하기

MMC (MultiMediaCard) 란?

1997년에 나온 플래시 메모리(Flash memory) 카드 표준

- ✓ 플래시 메모리? 전원이 공급되지 않아도 데이터가 보존되는 메모리 (e.g., SSD)
- SD 카드(Secure Digital Card)가 기준으로 하고 있는 표준
 - MMC와 SD 카드는 크기와 핀 배열이 유사, SD카드를 지원하는 장치에서 MMC 호환

일반적으로 MMC와 SD 카드라는 용어를 많이 혼용해서 사용

➤ 본 수업에서는 SD카드가 호환되는 ARM PL180 MMC 인터페이스 하드웨어를 사용합니다.



QEMU의 새 SoC에 MMC 장치 추가하기

```
...
#include "hw/sd/sd.h"
...
#define COMENTO_IRQ_MMC 2
...
#define COMENTO_BASE_MMC      0x09013000
...
static void create_mmc(const ComentoMachineState *vms) {
    hwaddr base = COMENTO_BASE_MMC;
    int irq = COMENTO_IRQ_MMC;
    DeviceState *dev;
    DriveInfo *dinfo;
    // ARM PL181 MMC 인터페이스 장치 추가
    dev = sysbus_create_varargs("pl181", base,
                                // 송신과 수신 위한 인터럽트 추가
                                qdev_get_gpio_in(vms->gic, irq),
                                qdev_get_gpio_in(vms->gic, irq + 1),
                                NULL);
    // QEMU의 -drive format=raw,file=<이미지 이름>,if=sd 옵션으로 지정된
    // SD 카드 이미지 정보 얻기
    dinfo = drive_get(IF_SD, 0, 0);
    if (dinfo) {
        DeviceState *card;
        card = qdev_new(TYPE_SD_CARD); // SD 카드 장치 추가
        qdev_prop_set_drive_err(card, "drive", blk_by_legacy_dinfo(dinfo),
                                &error_fatal);
        qdev_realize_and_unref(card, qdev_get_child_bus(dev, "sd-bus"),
                                &error_fatal); // SD 카드 삽입
    }
}
```

```
...
static void machcomento_init(MachineState *machine)
{
    ...
    create_mmc(vms);
}
```

<hw/arm/comento.c>

디바이스 트리와 커널 설정에 MMC 장치 추가하기

```
...
pl181@9013000 {
    compatible = "arm,pl181", "arm,primecell";
    clock-names = "apb_pclk";
    clocks = <&clock>;
    // 여기에 명시된 장치를 사용하여
    // MMC와 SD카드에 전원을 켜거나 끄므로 반드시 명시해야 함
    vmmc-supply = <&regulator>;
    // 읽기와 쓰기 각각에 대해 인터럽트 사용
    interrupts = <GIC_SPI 2 IRQ_TYPE_LEVEL_HIGH>,
                 <GIC_SPI 3 IRQ_TYPE_LEVEL_HIGH>;
    reg = <0x00 0x9013000 0x00 0x1000>;
};

// 전원을 공급하는 장치인 PMIC 명시
regulator: fixed {
    // 고정된 전압을 공급하는 장치
    compatible = "regulator-fixed";
    // 전원 공급 장치는 항상 켜져 있음을 명시
    regulator-always-on;
    // 전압의 조정 가능 범위를 명시
    // MMC와 SD카드의 전원 기본 전압은 3.3V
    // QEMU에서는 실제로 전압이 조정되지는 않으므로 최소/최대를 같게
    regulator-max-microvolt = <3300000>;
    regulator-min-microvolt = <3300000>;
    regulator-name = "3V3";
};
...
```

<arch/arm64/boot/dts/comento/comento.dts>

```
...
CONFIG_MMC=y
CONFIG_MMC_ARMMMC=y
// PMIC로 사용하는 regulator-fixed 관련 설정
CONFIG_REGULATOR=y
CONFIG_REGULATOR_FIXED_VOLTAGE=y
```

<arch/arm64/configs/comento_defconfig>

수정한 defconfig를 적용하기 위해
make comento_defconfig 명령어 사용

PMIC (Power Management Integrated Circuit) 란?

말 그대로, 전원을 관리하는 장치

- 전자 회로마다 다양한 전압을 요구함
 - USB 장치는 5V, SD 카드는 3.3V, 일부 센서는 1.8V, RAM은 1.35V로 다양
- 배터리를 사용하는 경우 전력의 소모가 작아야 함
 - 게임 등 CPU의 속도가 중요한 경우, CPU 클럭과 전압을 높임
 - 높은 클럭 일수록 손실되는 전력이 높으므로, 전압을 그에 맞춰 높여줘야 함
 - 현재 대기상태인 경우 CPU 속도가 중요하지 않으므로, CPU 클럭과 전압을 낮춤
- 전압을 목표한 대로 유지시켜 줌 - 전압 레귤레이터 (Voltage regulator)

SD 카드 이미지 생성하기 (1/3)

1. 목표로 하는 크기의 0으로 채워진 이미지 파일 생성

- `dd if=/dev/zero of=sdcard.img count=1 bs=64M`
 - 64MB 크기의 0으로 채워진 `sdcard.img` 생성

2. 이미지 파일에 파티션 정보를 추가

- `fdisk sdcard.img`
 - `n <엔터> p<엔터> 1<엔터>`
`<엔터> <엔터> w<엔터>`

✓ 파티션(Partition) 이란?

하나의 디스크를 여러 개의 드라이브로
분할해서 사용할 수 있는 기능

➤ 본 수업에서는 1개의 드라이브만 생성

```
Welcome to fdisk (util-linux 2.37.2).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Command (m for help) n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1): 1
First sector (2048-131071, default 2048): 
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-131071, default 131071): 

Created a new partition 1 of type 'Linux' and of size 63 MiB.

Command (m for help) w
The partition table has been altered.
Syncing disks.
```

SD 카드 이미지 생성하기 (2/3)

3. 이미지 파일을 Loop 디바이스로 설정

- 이미지 파일을 바로 마운트 불가, Loop 디바이스 상태에서 마운트 가능
- `sudo losetup -Pf --show sdcard.img`

```
mentoring@comento:~$ sudo losetup -Pf --show sdcard.img  
/dev/loop8
```

- 출력되는 디바이스 노드 파일이 설정된 loop 디바이스

4. Loop 디바이스의 첫 번째 파티션을 ext4 형식으로 포맷

- `sudo mkfs.ext4 <loop 디바이스 경로>p1`

```
mentoring@comento:~$ sudo mkfs.ext4 /dev/loop8p1  
mke2fs 1.46.5 (30-Dec-2021)  
Discarding device blocks: done  
Creating filesystem with 16128 4k blocks and 16128 inodes  
  
Allocating group tables: done  
Writing inode tables: done  
Creating journal (1024 blocks): done  
Writing superblocks and filesystem accounting information: done
```

SD 카드 이미지 생성하기 (3/3)

5. 포맷한 파티션과 빌드루트에서 생성된 이미지 동시에 마운트

- `mkdir mnt1 mnt2`
- `sudo mount -o loop <loop 디바이스 경로>p1 mnt1`
- `sudo mount -o loop <빌드루트 디렉토리>/output/images/rootfs.ext4 mnt2`

6. 마운트된 빌드루트 이미지의 내용을 마운트된 포맷 파티션으로 복사

- `sudo cp -R mnt2/* mnt1/.`

7. 마운트한 디렉토리를 모두 언마운트

- `sync; sudo umount mnt1 mnt2`

8. Loop 디바이스 해제

- `sudo losetup -d <loop 디바이스 경로>`

QEMU 실행시 SD 카드 사용 실습 (1/2)

```
<QEMU 디렉토리/build/qemu-system-aarch64 ₩  
-kernel <리눅스 디렉토리>/arch/arm64/boot/Image ₩  
-drive format=raw,file=sdcard.img,if=sd ₩  
-append "root=/dev/mmcblk0p1 console=ttyAMA0 rootwait"  
-dtb <리눅스 디렉토리>/arch/arm64/boot/dts/comento/comento.dtb ₩  
-qmp unix:/tmp/qmp.sock,server,nowait -nographic -M comento -m 1G -smp 4
```

- -drive 옵션으로 SD 카드 형태로 sdcard.img가 QEMU 내부에 전달됨
- -append로 전달되는 커널 파라미터에서 root는 rootfs의 위치인 mmcblk의 첫 번째 파티션 rootwait은 rootfs가 마운트될 때까지 기다림을 나타냄

QEMU 실행시 SD 카드 사용 실습 (2/2)

```
mmci-pl18x 9013000.pl181: mmc0: PL181 manf 41 rev0 at 0x09013000 irq 21,22 (pio)
```

...

```
Waiting for root device /dev/mmcblk0p1...
mmc0: host does not support reading read-only switch, assuming write-enable
mmc0: new SD card at address 4567
mmcblk0: mmc0:4567 QEMU! 64.0 MiB
mmcblk0: p1
EXT4-fs (mmcblk0p1): INFO: recovery required on readonly filesystem
EXT4-fs (mmcblk0p1): write access will be enabled during recovery
EXT4-fs (mmcblk0p1): recovery complete
EXT4-fs (mmcblk0p1): mounted filesystem 12744c1e-2456-4e4d-ad3e-ef49127bb448 ro
with ordered data mode. Quota mode: disabled.
VFS: Mounted root (ext4 filesystem) readonly on device 179:1.
devtmpfs: mounted
Freeing unused kernel memory: 640K
Run /sbin/init as init process
random: crng init done
EXT4-fs (mmcblk0p1): re-mounted 12744c1e-2456-4e4d-ad3e-ef49127bb448 r/w. Quota
mode: disabled.
Starting syslogd: OK
Starting klogd: OK
Running sysctl: OK
Seeding 256 bits and crediting
Saving 256 bits of creditable seed for next boot

Welcome to Buildroot
buildroot login:
```

- ARM PL181 장치가 PIO 방식으로 초기화
- 커널 파라미터 rootwait으로 rootfs 마운트될 때까지 대기
- -drive로 지정한 이미지가 SD 카드 형태로 삽입됨
- SD 카드 이미지의 첫 번째 파티션 마운트



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.

과제 안내

과제 의도

- 기존의 MMIO 하드웨어를 DMA를 사용하여 성능을 개선하여 본다.
- DMA가 하드웨어와 어떤 식으로 구성되는지 감을 잡을 수 있다.

필수 과제 내용

1. 지난주 과제에서 만들어 보았던 Logging 디바이스가 DMA를 지원하도록 변경한다.
 1. QEMU에 흐름제어를 위해 DMA와 연동하는 부분을 추가한다.
 2. 리눅스 디바이스 드라이버를 수정하여 DMA를 사용하도록 한다.
 3. 디바이스 트리를 그에 맞게 변경한다.
2. QEMU 디바이스 코드와 리눅스의 디바이스 드라이버, 디바이스 트리를 업로드하여 제출한다.

과제 안내

선택 과제 내용

1. memcpy와 동일하지만 DMA를 사용하는 커널 함수 dma_memcpy를 만들어 본다.

1. memcpy를 위해 사용할 수 있는 DMA 채널 얻어 오는 예시 코드

```
dma_cap_mask_t mask;  
dma_cap_zero(mask);  
dma_cap_set(DMA_MEMCPY, mask);  
[DMA 채널] = dma_request_channel(mask, NULL, NULL);
```

2. dmaengine_prep_slave_sg 대신 dmaengine_prep_dma_memcpy를 사용하여 함

```
[주소의 dma_addr_t] = dma_map_single([DMA 채널]->device->dev, [주소], [크기], DMA_BIDIRECTIONAL)  
dmaengine_prep_dma_memcpy([DMA 채널], [dst의 dma_addr_t], [src의 dma_addr_t], [크기],  
DMA_CTRL_ACK | DMA_PREP_INTERRUPT)
```

2. 별도의 커널 모듈을 만들고, 모듈 로드시에 dma_memcpy를 사용하는 예제를 구성한다.

3. 해당 커널 모듈의 코드를 업로드하여 제출한다.

기한 : 다음 주 월요일 오전 0시까지

다음 주 커리큘럼

GPIO 이해하기

새로운 GPIO 하드웨어 만들기

libgpiod 툴로 GPIO 테스트하기

드라이버에서 GPIO 사용하기

PWM 이해하기

